CardioIA — Fase 3

Relatório Técnico — Parte 1: Edge Computing e Resiliência Offline

> ⚠️ **AVISO CLÍNICO:** Este sistema é um protótipo acadêmico e **NÃO** substitui avaliação médica. Supervisionado por Dra. Fernanda Fassina (CRM-SP 169944).

Equipe (Grupo S — FIAP): Caroline de Castro Corrêa (RM567255), Enzo França Sader (RM566928), Lucas Hideki Oliveira Koyama (RM566925), Rodrigo Dias Figueiroa (RM567800) e Tiago Lindgren Curi (RM567016).

1. Introdução

O projeto CardioIA, desenvolvido no curso de Inteligência Artificial da FIAP, tem como objetivo explorar a aplicação de tecnologias inteligentes no contexto da saúde cardiovascular.

Na Fase 3, foi desenvolvido um protótipo de monitoramento contínuo utilizando um microcontrolador ESP32 simulado na plataforma Wokwi. A implementação demonstra a coleta periódica de sinais vitais, o processamento local de informações e a capacidade de continuar operando durante interrupções de conectividade.

O principal objetivo desta etapa é demonstrar o conceito de **Edge Computing**, no qual parte do processamento e do armazenamento ocorre próximo à origem dos dados, reduzindo a dependência de uma conexão permanente com serviços externos.

2. Componentes e tecnologias

O protótipo utiliza um ESP32, um sensor DHT22 e um botão de pressão (pushbutton).

O DHT22 realiza leituras simuladas de temperatura e umidade relativa do ambiente. O pushbutton representa um simulador manual de pulsos cardíacos, permitindo calcular uma estimativa de batimentos por minuto (BPM) a partir do intervalo entre acionamentos sucessivos.

O circuito foi desenvolvido na plataforma Wokwi e programado em C++, utilizando o ambiente Arduino e a biblioteca DHTesp. O mesmo firmware foi estendido na Parte 2 com Wi-Fi e MQTT (biblioteca PubSubClient); este relatório descreve a camada Edge, e a comunicação com a nuvem está em `relatorio_mqtt.md`.

Conexões utilizadas:

- DHT22: alimentação de 3,3 V, sinal de dados conectado ao GPIO 15 e aterramento GND.
- Pushbutton: conectado ao GPIO 18 e ao GND, com configuração `INPUT_PULLUP`.

3. Funcionamento do monitoramento

O ESP32 executa continuamente a leitura dos componentes conectados.

A coleta de temperatura e umidade acontece em intervalos de aproximadamente dois segundos. Paralelamente, o sistema monitora o botão responsável pela simulação dos pulsos cardíacos.

O código utiliza a função `millis()` para controlar intervalos temporais sem interromper completamente o processamento, evitando os bloqueios causados pela utilização prolongada de `delay()`.

A estimativa de BPM utiliza a fórmula:

BPM = 60.000 / intervalo entre pulsos, em milissegundos.

A implementação também possui uma rotina de *debounce* para reduzir múltiplas detecções provocadas por oscilações do botão e um mecanismo de expiração da estimativa quando não há novos pulsos durante quatro segundos.

Quando não existe uma estimativa válida, o programa identifica o BPM como indisponível, evitando interpretá-lo como medição fisiológica real.

4. Estratégia de resiliência offline

Para simular interrupções de conectividade, o programa utiliza uma variável booleana. Na versão inicial (somente Serial) ela se chamava `wifiConectado`; no firmware atual, que também faz a conexão real ao Wi-Fi do Wokwi e ao broker, o mesmo papel é exercido por `envioHabilitado`, alternada pelos comandos `f` e `o` do Monitor Serial. O dispositivo **inicia offline** para evidenciar a resiliência.

Quando seu valor é `false`, o sistema entra em modo offline. Nesse estado, a coleta dos sensores continua normalmente, mas as leituras não são transmitidas.

Os registros são armazenados temporariamente em uma **fila circular em memória RAM**, com capacidade fixa de 1.800 leituras.

Considerando um intervalo aproximado de dois segundos por coleta, essa capacidade corresponde a cerca de uma hora de dados acumulados.

Cada registro contém um identificador sequencial, o tempo decorrido desde o início da execução, temperatura, umidade, BPM estimado e um indicador de validade do BPM.

Quando a fila atinge sua capacidade máxima, a leitura mais antiga é descartada para permitir a entrada de uma nova leitura. O sistema também contabiliza o número de descartes, possibilitando identificar perdas durante períodos prolongados sem conexão.

Essa política de armazenamento limitado foi escolhida para demonstrar uma estratégia de gestão de memória em dispositivos embarcados.

5. Reconexão e sincronização

Quando a conexão simulada é restaurada (comando `o`), a variável de conectividade passa para `true`.

Nesse momento, o ESP32 inicia a sincronização dos registros pendentes, processando as leituras na ordem em que foram coletadas.

A implementação limita o processamento a dez registros por passagem do loop principal, evitando concentrar toda a execução na rotina de sincronização.

Na versão inicial (Parte 1), a transmissão era representada por mensagens JSON exibidas no Monitor Serial, conforme o enunciado. No firmware atual, a mesma rotina publica cada leitura em JSON no tópico MQTT (Parte 2).

A remoção de um registro da fila ocorre somente após a função de transmissão retornar sucesso, isto é, após `mqtt.publish()` aceitar a mensagem. Como a publicação usa QoS 0, esse sucesso indica apenas que a biblioteca enviou a mensagem, e não que o broker a recebeu; esse limite é discutido em `relatorio_mqtt.md`.

Dimensionamento da fila. Cada registro ocupa 24 bytes (dois `unsigned long`, três `float` e um `bool`, com alinhamento), então 1.800 registros usam cerca de 42 KB de RAM. Na compilação do firmware completo para ESP32 (PlatformIO, placa `esp32dev`), o uso total de RAM estática foi de cerca de 90 KB dos 320 KB disponíveis (27,6%), já incluindo a pilha Wi-Fi/MQTT e a fila. A capacidade de uma hora de coleta foi escolhida como compromisso entre autonomia offline e memória: para um monitor vestível, uma hora cobre quedas curtas de conectividade (deslocamentos, elevadores, instabilidade de rede) sem consumir parte relevante dos recursos do ESP32. Acima disso, a política é descartar a leitura mais antiga e manter as mais recentes, que são as mais relevantes para o monitoramento.

6. Testes realizados e resultados

O funcionamento do protótipo foi verificado no ambiente Wokwi. Os testes de coleta e de resiliência abaixo foram feitos na versão inicial do firmware (transmissão simulada pelo Serial); a verificação com publicação MQTT e dashboard está em `relatorio_mqtt.md` e nas capturas de `evidencias/`.

Inicialmente, foi confirmada a leitura periódica do DHT22, com valores simulados de temperatura e umidade apresentados no Monitor Serial.

Em seguida, foram realizados acionamentos do pushbutton, confirmando a detecção dos pulsos e o cálculo da estimativa de BPM.

Para validar a resiliência, o dispositivo foi mantido em estado offline durante múltiplos ciclos de coleta.

O Monitor Serial demonstrou o armazenamento de três leituras pendentes:

- Leitura 1: aproximadamente 2.000 ms.
- Leitura 2: aproximadamente 4.000 ms.
- Leitura 3: aproximadamente 6.000 ms.

Após a restauração da conexão, o sistema exibiu a sincronização das três leituras, mantendo seus identificadores e a ordem original.

Em seguida, uma nova leitura foi coletada no estado online e transmitida normalmente.

Esses resultados demonstram, no ambiente simulado, a continuidade da coleta durante períodos offline e o esvaziamento da fila após o restabelecimento da conectividade.

7. Limitações e considerações de segurança

O protótipo apresenta limitações decorrentes de seu caráter acadêmico.

O armazenamento em memória RAM é volátil. Portanto, as leituras são perdidas quando a simulação é encerrada ou o dispositivo é reiniciado.

Não foi implementado armazenamento persistente em SPIFFS, considerando as limitações do ambiente de simulação e o caráter opcional desse requisito.

Além disso, o DHT22 mede temperatura e umidade ambientais, não temperatura corporal. Os pulsos cardíacos são simulados por acionamentos manuais e não representam medições obtidas de pacientes.

A variável de conectividade é um interruptor de teste (`f`/`o`): ela simula a queda e a volta da rede, mas não detecta falhas reais de Wi-Fi. A confiabilidade da entrega ao broker também é limitada pelo QoS 0 (ver `relatorio_mqtt.md`).

Outra limitação é que os limiares de alerta são avaliados apenas no dashboard (camada Fog). Com o dispositivo offline, nenhum alerta é emitido: em um monitor real, o alerta local (por exemplo, LED ou buzzer no próprio dispositivo) seria parte essencial do Edge Computing.

Para uma aplicação médica real, seriam necessários sensores adequados e validados, armazenamento persistente, proteção de dados, comunicação criptografada, autenticação, mecanismos confiáveis de confirmação de entrega e avaliação de segurança clínica.

8. Conclusão

A implementação da Parte 1 da Fase 3 demonstrou os principais conceitos de Edge Computing aplicados a um protótipo de monitoramento cardiovascular simulado.

O sistema conseguiu realizar leituras periódicas, calcular uma estimativa de BPM, armazenar informações temporariamente durante uma desconexão e sincronizar os registros após a recuperação da conectividade.

Essa base foi então integrada a um broker MQTT e a um dashboard Node-RED com alertas automáticos, descritos no relatório da Parte 2.

Referência do protótipo

Projeto no Wokwi: https://wokwi.com/projects/477362937220720641

Repositório: https://github.com/Grupo-S-faculdade-FIAP/cardioIA

Código: `fase3/esp32/sketch.ino` · **Evidências:** `fase3/docs/evidencias/` · **Relatório da Parte 2:** `relatorio_mqtt.md`